SIDDHARTHA ACADEMY OF HIGHER EDUCATION

(DEEMED TO BE UNIVERSITY)

VIJAYAWADA – 520007

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

HOME ASSIGNMENT ON

COURSE MATERIAL SYSTEM

FOR

OBJECT ORIENTED PROGRAMMING THROUGH JAVA

SUBMITTED BY

Roll Number	:	25EU02105
Name of Student	:	P.V.S.Bhavya Samhitha
Section	:	III-AIML B
Department	:	AI&ML

Academic Year: 2026–2027

Question
Develop a Course Material package using Course and Material classes. Use an interface and handle missing file errors.

1. Aim

To develop a Java package named `coursematerial` containing the classes `Course` and `Material` and the interface `Viewable`, demonstrating package organization, interface implementation and the handling of missing file errors (`FileNotFoundException`) using `throws`, `try-catch` and try-with-resources.

2. Introduction

A course has many study materials such as lecture notes, presentation slides and lab manuals, and each material is stored in a file. When a student opens the course, some of the files may be missing because they were deleted, renamed, moved or never uploaded. A good program must not crash when this happens. It must report which file is missing and carry on with the remaining materials.

In this program the package `coursematerial` contains the interface `Viewable` (which describes what every material must be able to do), the class `Material` (which implements the interface and reads its file) and the class `Course` (which keeps a list of materials and opens them one by one). The main class `CourseApp` is outside the package and imports these classes. Packages keep related classes together and avoid name conflicts, the interface separates what a material does from how it does it, and exception handling makes the program reliable when files are missing.

Three materials are used in the demonstration. Two of the files exist and one, `dbms_slides.txt`, is deliberately not created, so that the error handling can be observed in the output.

3. Objectives

- To create a package `coursematerial` with `Viewable`, `Material` and `Course`.
- To create an interface with a constant and abstract methods and to implement it in `Material`.
- To read a text file using `BufferedReader` and `FileReader`.
- To handle a missing file using `FileNotFoundException` with `throws` and `try-catch`.
- To use try-with-resources so that the file is closed automatically.
- To import the package classes into a main class outside the package.

4. System Requirements and Folder Structure

Item	Requirement
Language	Java (JDK 8 or any later version)

Item	Requirement
Compiler and runtime	javac and java commands of the JDK
Editor / IDE	Notepad, VS Code, IntelliJ IDEA, Eclipse or NetBeans
Input files	java_notes.txt and lab_manual.txt in the run folder; dbms_slides.txt is intentionally absent
Operating system	Windows, Linux or macOS

Create a folder named `CourseProject` and arrange the files as shown below. The package folder must have exactly the same name as the package.

```
CourseProject/
  |-- coursematerial/
  |     |-- Viewable.java
  |     |-- Material.java
  |     |-- Course.java
  |-- CourseApp.java
  |-- java_notes.txt
  |-- lab_manual.txt
```

Compile with `javac coursematerial/*.java CourseApp.java` and run with `java CourseApp` from inside the `CourseProject` folder. The text files are opened using relative names, so the program must be run from this folder.

5. Theory and Concepts Used

5.1 Package

A package is a collection of related classes and interfaces. The statement `package coursematerial;` must be the first statement in every source file of the package. Packages avoid name conflicts between classes written by different programmers, give access control and make large programs easier to maintain.

5.2 Package Organization and Directory Structure

The classes of a package are stored in a folder with the same name as the package. Here `Viewable.java`, `Material.java` and `Course.java` are stored in the folder `coursematerial`. The main class `CourseApp.java` is stored in the parent folder, and the compiler creates the class files next to the source files.

5.3 Interface

An interface is declared with the `interface` keyword. The variables of an interface are automatically `public`, `static` and `final`, and its methods are `public` and `abstract`. A class uses the `implements` keyword and must provide a body for every method of the interface. `Viewable` declares `getTitle()` and `view()` and the constant `MAX_PREVIEW_LINES`; `Material` implements

them. `Course` keeps an array of `Viewable` references, so it works with any class that implements the interface, not only with `Material`.

5.4 import Statement and Access Modifiers

The `import` statement makes the classes of another package available. Only `public` classes and `public` members can be used from outside the package, so `Viewable`, `Material` and `Course`, their constructors and their methods used by `CourseApp` are declared `public`. The fields remain `private`, which provides encapsulation.

5.5 File Handling Classes

`FileReader` opens a text file for reading. `BufferedReader` wraps a reader and provides the method `readLine()`, which returns one line at a time and returns `null` at the end of the file. The constructor of `FileReader` throws a `FileNotFoundException` if the file does not exist, is a directory or cannot be opened for reading.

5.6 Checked Exceptions and the Exception Hierarchy

Class	Meaning
Throwable	Parent of all errors and exceptions
Exception	Conditions that a program should handle; checked exceptions extend it
IOException	Checked exception for input and output failures
FileNotFoundException	Subclass of IOException; the file does not exist or cannot be opened

A checked exception must either be handled with `try-catch` or declared with `throws`; otherwise the program does not compile. Because `FileNotFoundException` is a subclass of `IOException`, the catch block for `FileNotFoundException` must be written before the catch block for `IOException`; the other order is a compile-time error.

5.7 throw, throws and try-catch

`throw` raises an exception object, `throws` declares in the method heading that a method may raise an exception, and `try-catch` handles it. In this program `Material.view()` declares `throws FileNotFoundException` and throws a new exception with a clear message. `Course.openAll()` catches it. Because the `try-catch` is inside the loop of `openAll()`, a missing file affects only that one material and the loop continues with the next.

5.8 try-with-resources

A resource that is declared in the parentheses after `try` is closed automatically when the block ends, whether it ends normally or because of an exception. This removes the need for a `finally` block that

closes the file and prevents resource leaks. The resource must implement `AutoCloseable`, which `BufferedReader` does.

5.9 Constants in an Interface

`MAX_PREVIEW_LINES` is written without any modifier but is automatically `public static final`. It is available to `Material` directly by its name, and any other class can use it as `Viewable.MAX_PREVIEW_LINES`. Using one constant in the interface keeps the preview length the same for every kind of material.

6. Class Description

Type	Fields / constants	Important methods
Viewable (interface)	MAX_PREVIEW_LINES = 3	getTitle(), view() throws FileNotFoundException
Material	title, fileName	getTitle(), view() with try-with-resources
Course	code, name, materials[], count	addMaterial(), openAll(), getMaterialCount()
CourseApp	None	main()

`Material` implements `Viewable`. `Course` has an array of `Viewable` references and therefore depends only on the interface. `CourseApp` creates the `Material` objects and stores them in `Viewable` references, which is called programming to an interface.

7. Algorithm

1. Start the program.
2. Create the package `coursematerial` and the interface `Viewable` with the constant MAX_PREVIEW_LINES and the methods `getTitle()` and `view()`.
3. Create the class `Material` implementing `Viewable`. In `view()`, open the file with `FileReader` inside try-with-resources, print at most three lines, and throw a `FileNotFoundException` with a clear message if the file is missing.
4. Create the class `Course` with an array of `Viewable`, `addMaterial()` and `openAll()`. In `openAll()`, call `view()` on each material inside `try-catch` and count the missing files.
5. Create the text files `java_notes.txt` and `lab_manual.txt` in the run folder. Do not create `dbms_slides.txt`.
6. In `CourseApp`, import the package classes, create a `Course` and three `Material` objects and add them to the course.
7. Call `openAll()` and display the number of materials, the number opened and the number missing.

8. Stop.

8. Complete Java Program

Save the files as shown below inside the folder `CourseProject`.

File 1: coursematerial/Viewable.java

```
package coursematerial;

import java.io.FileNotFoundException;

public interface Viewable {
    int MAX_PREVIEW_LINES = 3;      // constant: public static final

    String getTitle();
    void view() throws FileNotFoundException;
}
```

File 2: coursematerial/Material.java

```
package coursematerial;

import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;

public class Material implements Viewable {
    private String title;
    private String fileName;

    public Material(String title, String fileName) {
        this.title = title;
        this.fileName = fileName;
    }

    @Override
    public String getTitle() {
        return title;
    }

    @Override
    public void view() throws FileNotFoundException {
        // try-with-resources closes the file automatically
        try (BufferedReader br = new BufferedReader(new FileReader(fileName))) {
            System.out.println("Material: " + title + " (" + fileName + ")");
            String line;
            int count = 0;
            while ((line = br.readLine()) != null && count < MAX_PREVIEW_LINES) {
                System.out.println("  " + line);
                count++;
            }
        } catch (FileNotFoundException e) {
            throw new FileNotFoundException("File not found: " + fileName);
        } catch (IOException e) {
```

```
            System.out.println("Could not read " + fileName + ": " + e.getMessage());
        }
    }
}
```

File 3: coursematerial/Course.java

```
package coursematerial;

import java.io.FileNotFoundException;

public class Course {
    private String code;
    private String name;
    private Viewable[] materials = new Viewable[10];
    private int count = 0;

    public Course(String code, String name) {
        this.code = code;
        this.name = name;
    }

    public void addMaterial(Viewable m) {
        if (count < materials.length) {
            materials[count++] = m;
        }
    }

    // Opens every material; a missing file does not stop the others
    public int openAll() {
        System.out.println("Course: " + code + " - " + name);
        int missing = 0;
        for (int i = 0; i < count; i++) {
            try {
                materials[i].view();
            } catch (FileNotFoundException e) {
                System.out.println("Missing material \"" + materials[i].getTitle()
                        + "\": " + e.getMessage());
                missing++;
            }
        }
        return missing;
    }

    public int getMaterialCount() { return count; }
}
```

File 4: CourseApp.java

```
import coursematerial.Course;
import coursematerial.Material;
import coursematerial.Viewable;

public class CourseApp {
    public static void main(String[] args) {
        Course course = new Course("CS204", "OOP through Java");
```

```
        Viewable m1 = new Material("Java Notes", "java_notes.txt");
        Viewable m2 = new Material("DBMS Slides", "dbms_slides.txt");   // file does not exist
        Viewable m3 = new Material("Lab Manual", "lab_manual.txt");

        course.addMaterial(m1);
        course.addMaterial(m2);
        course.addMaterial(m3);

        System.out.println("COURSE MATERIAL SYSTEM");
        int missing = course.openAll();

        System.out.println("\nTotal Materials: " + course.getMaterialCount());
        System.out.println("Opened Successfully: " + (course.getMaterialCount() - missing));
        System.out.println("Missing Files: " + missing);
    }
}
```

File 5: java_notes.txt

```
Java is object oriented.
A class is a blueprint of objects.
Inheritance reuses code.
Polymorphism has many forms.
```

File 6: lab_manual.txt

```
Lab 1: Classes and Objects
Lab 2: Inheritance
```

The file `dbms_slides.txt` is intentionally not created.

9. Explanation of the Program

Viewable interface

`Viewable` defines the constant MAX_PREVIEW_LINES (3) and declares `getTitle()` and `view()`. The method `view()` declares `throws FileNotFoundException`, so every implementing class is expected to report a missing file to its caller instead of hiding it.

Material class

`Material` stores a title and a file name as private fields and implements `Viewable`. `view()` opens the file inside try-with-resources and prints at most MAX_PREVIEW_LINES lines, each indented by two spaces. If the file is missing, the first catch block throws a new `FileNotFoundException` that carries the message "File not found" followed by the file name; this gives a clean message that is the same on every operating system. The second catch block handles any other `IOException` that occurs while the file is being read.

Course class

`Course` keeps a `Viewable` array and a count. `addMaterial()` adds a material if there is space. `openAll()` calls `view()` for each material inside a `try-catch`. When a `FileNotFoundException` is caught, it prints the title of the missing material together with the message, increases the missing counter and continues with the next material. It returns the number of missing files.

CourseApp class

`CourseApp` imports the three classes of the package, creates three `Material` objects (stored in `Viewable` references) and adds them to a `Course`. It calls `openAll()` and prints the total number of materials, the number opened successfully and the number of missing files.

10. Sample Output

```
COURSE MATERIAL SYSTEM
Course: CS204 - OOP through Java
Material: Java Notes (java_notes.txt)
  Java is object oriented.
  A class is a blueprint of objects.
  Inheritance reuses code.
Missing material "DBMS Slides": File not found: dbms_slides.txt
Material: Lab Manual (lab_manual.txt)
  Lab 1: Classes and Objects
  Lab 2: Inheritance

Total Materials: 3
Opened Successfully: 2
Missing Files: 1
```

11. Output Explanation

The first material, Java Notes, is found. The file has four lines, but only the first three are printed because MAX_PREVIEW_LINES is 3.

The second material, DBMS Slides, refers to `dbms_slides.txt`, which does not exist. The `FileReader` constructor fails, `Material.view()` throws a `FileNotFoundException` with the message "File not found: dbms_slides.txt", and `Course.openAll()` catches it, prints the message and increases the missing counter.

The program does not stop. The third material, Lab Manual, is found and both of its lines are printed. The summary therefore shows 3 materials, 2 opened successfully and 1 missing file.

12. Test Cases

Situation	Expected behaviour	Summary line
All three files exist	All materials are previewed, no message about missing files	Missing Files: 0

Situation	Expected behaviour	Summary line
dbms_slides.txt is missing (demonstration)	Message for DBMS Slides; the others are shown	Missing Files: 1
All three files are missing	Three missing-file messages, no crash	Missing Files: 3
A file exists but is empty	Only the material heading is printed	Counted as opened
A file has more than 3 lines	Only the first 3 lines are previewed	Counted as opened
Program run from another folder	Relative file names are not found, so all are reported missing	Missing Files: 3

13. Advantages and Limitations

Advantages

- The package keeps all the course-related classes together and avoids name conflicts.
- Because `Course` depends on the `Viewable` interface, new kinds of material such as videos or web pages can be added without changing `Course`.
- The program does not crash when a file is missing; it reports the problem and continues.
- Try-with-resources closes every opened file automatically, even when an exception occurs.

Limitations

- The course can hold at most ten materials because a fixed-size array is used.
- Only plain text files are supported and only a short preview is shown.
- File names are relative, so the program must be run from the correct folder.

14. Possible Enhancements

- Use an `ArrayList<Viewable>` so that any number of materials can be added.
- Add a custom exception such as `MaterialNotFoundException` that extends `Exception`.
- Check the file with `java.io.File.exists()` before opening it and log the missing files to a report.
- Add other implementations of `Viewable` for PDF files, videos and links.

15. Conclusion

The Course Material System was implemented using the package `coursematerial` containing the `Viewable` interface and the `Material` and `Course` classes. The program demonstrates package organization, interface implementation, access modifiers, file reading with try-with-resources and the handling of missing file errors with `FileNotFoundException`, so that one missing file does not stop the remaining materials.

Result: The program displays a preview of each available material, reports the missing material and displays the number of materials opened and missing.